2026-5-19 C++priority_queue--底层实现

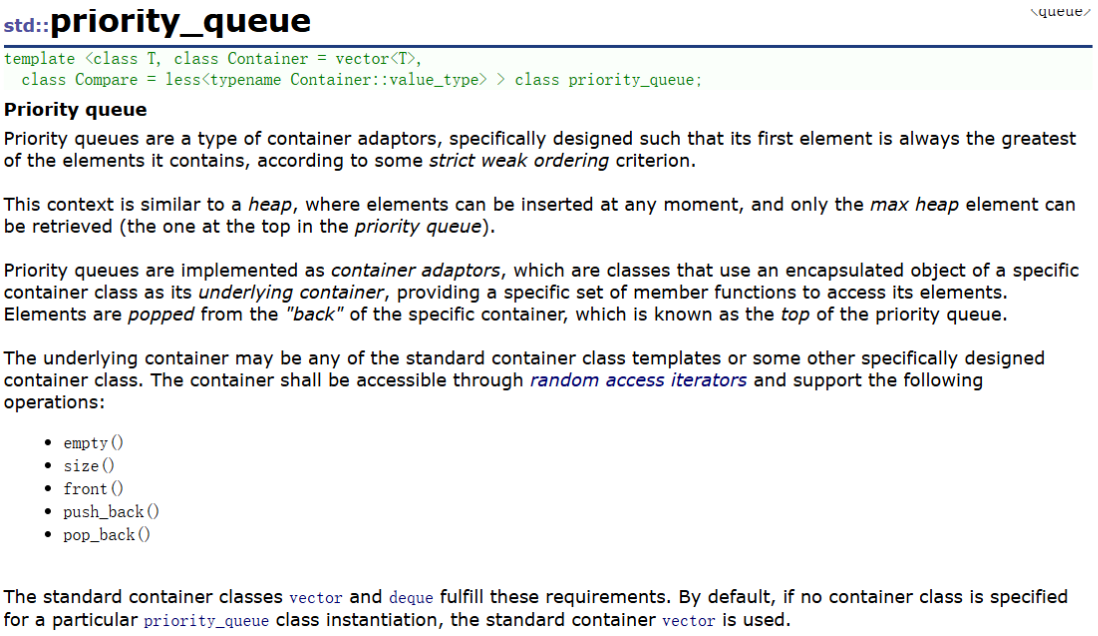
- 1. 引言：什么是priority_queue
  - 1.1 基本概念
  - 1.2 底层结构
  - 1.3 比较器的默认行为
- 2. priority_queue的核心实现
  - 2.1 成员变量与默认构造
    - 2.1.1 结构框架与默认构造
    - 2.1.2 迭代器区间构造
  - 2.2 push插入数据
  - 2.3 pop删除数据
    - 2.4 top / empty / size
- 3. 适配器模式与priori_queue的设计
  - 3.1 什么是适配器模式
  - 3.2 priority_queue的适配器设计
- 4. 仿函数
  - 4.1 什么是仿函数?
  - 4.2 仿函数的用法
    - 4.2.1 作为比较器，控制容器或者算法的行为
    - 4.2.2 作为判断条件或者作为转换规则
    - 4.2.3 处理指针时自定义比较逻辑
  - 4.3 仿函数存在的意义

1. 引言：什么是priority_queue

1.1 基本概念

`priority_queue` 是 C++ 标准库中的一个**容器适配器**。它提供的是一种**优先级队列**的数据结构语义：

- 插入元素的时间复杂度为 O(log N)
- 删除元素的时间复杂度为 O(log N)
- 获取最高优先级元素的时间复杂度为 O(1)
  核心特性：每次 `pop` 出来的，都是当前队列中**优先级最高**的元素。



std::priority_queue <queue>

```
template <class T, class Container = vector<T>,
  class Compare = less<typename Container::value_type> > class priority_queue;
```

Priority queue

Priority queues are a type of container adaptors, specifically designed such that its first element is always the greatest of the elements it contains, according to some *strict weak ordering* criterion.

This context is similar to a *heap*, where elements can be inserted at any moment, and only the *max heap* element can be retrieved (the one at the top in the *priority queue*).

Priority queues are implemented as *container adaptors*, which are classes that use an encapsulated object of a specific container class as its *underlying container*, providing a specific set of member functions to access its elements. Elements are *popped* from the *"back"* of the specific container, which is known as the *top* of the priority queue.

The underlying container may be any of the standard container class templates or some other specifically designed container class. The container shall be accessible through *random access iterators* and support the following operations:

- empty()
- size()
- front()
- push_back()
- pop_back()

The standard container classes vector and deque fulfill these requirements. By default, if no container class is specified for a particular priority_queue class instantiation, the standard container vector is used.

1.2 底层结构

priority_queue底层默认使用std::vector作为容器，并在其之上维护一个堆结构：

- 大堆：父节点 ≥ 子节点 → `top()` 返回最大值
- 小堆：父节点 ≤ 子节点 → `top()` 返回最小值
  堆的性质通过两个核心操作来维护：**向上调整（插入时）** 和 **向下调整（删除时）**。
  但是priority_queue是反过来的：
  默认传小于less仿函数，大的优先级高；传大于greater仿函数，小的优先级高。因此我们在实现的过程中也要依照次风格。

1.3 比较器的默认行为

`priority_queue` 有一个可选的模板参数 `Compare`，用于定义元素之间的优先级比较规则：

- 如果不显式提供，默认使用 `std::less<T>`
- `std::less<T>` 是一个**仿函数**，内部调用 `operator<`
- 在默认 `std::less` 下，`priority_queue` 表现为**大堆**（最大值在堆顶）

如果你需要小堆，可以显式传入 `std::greater<T>` 或自定义仿函数。

> 注意：**比较器不是“必须写的”，它有默认行为。** 是否显式提供，取决于你是否需要改变默认的排序规则。我们下面会对比较器进行显示构造来演示仿函数和比较器的行为。

2. priority_queue的核心实现

2.1 成员变量与默认构造

2.1.1 结构框架与默认构造

```
template<class T, class Container = vector<T>, class Compare = Less<T>>
//Container，Compare 是我们对库里面的结构进行了显示构造便于我们理解底层结构
class priority_queue {
public:
    priority_queue() = default;  // 编译器生成默认构造
private:
    Container _con;               // 底层容器
};
```

这里引出Container，Compare我们下面的章节进行详细的解释

2.1.2 迭代器区间构造

构造结构：

```
template<class InputIterator>
priority_queue(InputIterator first, InputIterator last)
    :_con(first, last)
{
    //给一个迭代器区间构造优先队列的底层是堆结构
    //向下调整建队构造
    for (int i = (_con.size() - 1 - 1) / 2; i >= 0; i--) {
        adjust_down(i);
    }
}
```

因为存储底层结构是vector，逻辑结构是堆，所以说我们在构造的过程中就涉及到了建堆：(我们选择向下调整建堆，时间效率比向上建队高，具体原因这里不做阐述)
建堆时间复杂度O（N)。
向下调整算法：

```
void adjust_down(size_t parent) {
    Compare com;
    size_t child = parent * 2 + 1;
    while (child < _con.size()) {
        if (child + 1 < _con.size() && com(_con[child], _con[child + 1])) {
```

```
            child++;
        }
        // 如果 父节点优先级 < 孩子节点优先级，则交换
        // 在大堆中：parent < child -> com(parent, child) 为 true -> 交换
        // 在小堆中：parent > child -> com(parent, child) 为 true -> 交换
        if (com(_con[parent], _con[child])) {
            swap(_con[parent], _con[child]);
            parent = child;
            child = parent * 2 + 1;
        }
        else {
            break;
        }
    }
}
```

2.2 push插入数据

push逻辑：

```
void push(const T& x) {
    _con.push_back(x);
    adjust_up(_con.size() - 1);
}
```

插入数据并且需要维持堆结构，我们需要将插入的数据进行向上调整：

```
void adjust_up(size_t child) {
    Compare com;
    size_t parent = (child - 1) / 2;
    while (child > 0) {

        //if (_con[parent] < _con[child])
        if(com(_con[parent],_con[child])){
            swap(_con[parent], _con[child]);//调用的库里面的函数
            child = parent;
            parent = (child - 1) / 2;
        }
        else {
            break;
        }
```

```
    }
}
```

2.3 pop删除数据

```
void pop() {
    assert(!empty());
    std::swap(_con[0], _con[_con.size() - 1]);
    _con.pop_back();
    adjust_down(0);
}
```

2.4 top / empty / size

```
const T& top() const { return _con[0]; }
bool empty() const { return _con.empty(); }
size_t size() const { return _con.size(); }
```

对于上述的底层构造我们引出适配器模式和仿函数的使用，下面我们来详细解释一下👇👇👇

3. 适配器模式与priori_queue的设计

3.1 什么是适配器模式

适配器模式：把一个已有的东西包装一下，换个接口或者行为再使用。
（实际上就像手机充电线一样，有不同的接口来适配不同的手机。）
容器适配器：把底层容器包装成另一种数据结构。在这里说人话也就是将容器类型引入到模板里面，让你可以任意调用已有的底层容器的接口。

3.2 priority_queue的适配器设计

也就是：

```
template<class T, class Container = vector<T>, class Compare = Less<T>>
//Container让我们可以很轻松的更改priority_queue的底层存储逻辑
//这里只是给一个缺省值我们可以显示给的
class priority_queue {
```

```
    Container _con;
};
```

- `Container`：底层容器类型（默认 `vector<T>`）
- `Compare`：比较仿函数类型（默认 `Less<T>`，对应大堆）
  意义：模板适配器参数的缺省值让用户在使用时更简洁，同时保留了高度可定制性

4. 仿函数

4.1 什么是仿函数?

仿函数实际上就是将operator()进行了重载（我感觉可以类比一下operator[]的重载），让类和结构体可以像函数一样被调用。
仿函数的定义：

```
template<class T>
class Less {
public:
    bool operator()(const T& x, const T& y) {
        return x < y;
    }
};
template<class T>
class Greater {
public:
    bool operator()(const T& x, const T& y) {
        return x > y;
    }
};
```

4.2 仿函数的用法

4.2.1 作为比较器，控制容器或者算法的行为

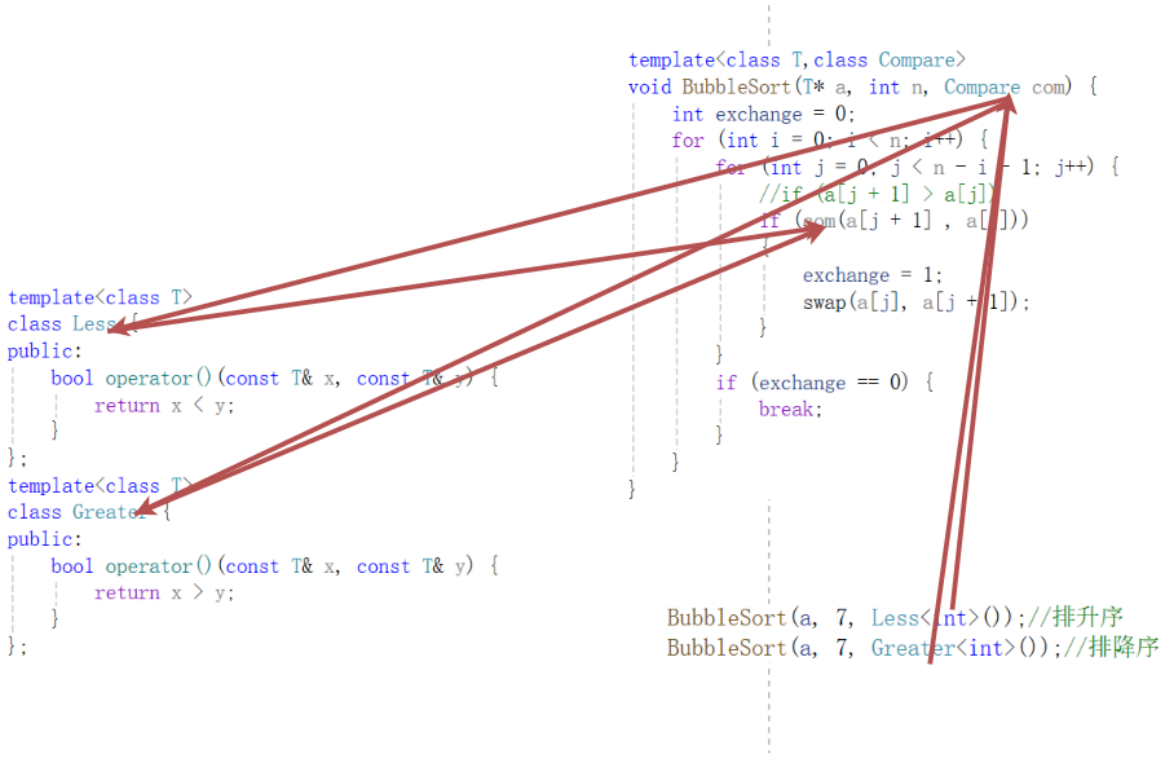
实际上就是将容器的规则或者算法的规则作为参数传入，让容器或者算法的行为可以定制。
代码示例1：

```
int main() {
//默认使用 Less → 大堆
```

```
    ZL::priority_queue<int> pq1;
//显示指定Greater->建大堆
    ZL::priority_queue<int, vector<int>, Greater<int>> pq2;
    pq1.push(100);
    pq1.push(5);
    pq1.push(200);
    pq1.push(10);
    while (!pq1.empty()) {
        cout << pq1.top() << " ";
        pq1.pop();
    }
    return 0;

}
```

示例代码2：




```
template<class T, class Compare>
void BubbleSort(T* a, int n, Compare com) {
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n - i - 1; ++j) {
            if (com(a[j + 1], a[j]))   // 比较规则由仿函数决定
                swap(a[j], a[j + 1]);
        }
    }
}

int main() {
```

```
    int a[] = { 1,3,4,5,6,3,2 };
    //让类可以像函数一样调用
    BubbleSort(a, 7, Less<int>());//排升序
    BubbleSort(a, 7, Greater<int>());//排降序
    for (auto ch : a) {
        cout << ch;//自动迭代不用++
    }
    cout << endl;
}
```

4.2.2 作为判断条件或者作为转换规则

示例代码1：查找第一个偶数（作为判断条件）

```
struct OP1 {
    bool operator()(int x) {
        return x % 2 == 0;
    }
};
int main() {
    int a[] = { 1,3,2,9,1,3,4,5 };
    //查找第一个偶数
    auto it = find_if(a, a + 7, OP1());
    cout << *it << endl
    return 0;
}
```

示例代码2：将偶数都乘二（作为转换条件）

```
struct OP2 {
        int operator()(int x) {
            if (x % 2 == 0)
                return x * 2;
            else
                return x;
        }

};
int main() {
    //transform 是 C++ 标准库中的一个算法，用于对容器中的每个元素执行某种操作，并将结果存储到另一个位置。
    //它属于 <algorithm> 头文件。
```

```
    transform(v.begin(), v.end(), v.begin(), OP2());
    for (auto& e : v) {
        cout << e << " ";
    }
    cout << endl;
    return 0;
}
```

4.2.3 处理指针时自定义比较逻辑

下面截取日期代码的一部分做演示：

```
//struct PDateLess
//{
//  bool operator()(const Date* p1, const Date* p2)//仿函数
//  {
//  return *p1 < *p2;//这里能直接对存储地址解引用对比出谁的日期大是因为
//在日期结构体中对<运算符进行了重载
//  }
int main() {
    ZL::priority_queue<Date*, vector<Date*>, PDateLess> q1;
    q1.push(new Date(2018, 10, 29));
    q1.push(new Date(2018, 10, 28));
    q1.push(new Date(2018, 10, 30));

    while (!q1.empty()) {
        cout << *q1.top() << " ";
        q1.pop();
    }
    cout << endl;
    return 0;
}
```

这里自定义仿函数的原因是：指针的默认比较是按地址大小，而不是按 `Date` 对象的内容。通过仿函数可以“纠正”这个行为（也就是解引用获得存储在该地址里的内容）。

4.3 仿函数存在的意义

仿函数的存在实际上是C++想要摒弃函数指针的使用，仿函数提供了比函数指针更优的替代方案，在现代 C++ 中优先推荐使用仿函数或 lambda。

问题	没有仿函数时的痛点	有仿函数时的解决方式
算法需要多种行为	写多个函数或用函数指针，效率低且不能内联	把行为封装成仿函数，编译期内联，无额外开销
需要携带状态	函数无法记住之前的调用信息	仿函数可以有成员变量，在多次调用间保持状态
类型安全	函数指针类型检查弱，容易传错	仿函数作为类类型，类型检查更严格
与模板结合	函数指针作为模板参数不自然	仿函数是类型，作为模板参数使用方便
复杂规则封装	难以复用	一个仿函数类可以在多个地方复用